Which of the following statements about AlexNet, Krizhevsky et al. 2012, are correct?

a. AlexNet uses input images with a resolution of 224 × 224 pixels and three channels.

b. AlexNet is a deep convolutional network with more than 1 hidden layer.

c. AlexNet does not have fully-connected layers.

d. One of the most important components of AlexNet is BatchNorm.

e. AlexNet uses strided convolutions instead of max-pooling for down-sampling.

a, b correct

AlexNet uses RGB image crops, commonly described as **224 × 224 × 3.** AlexNet did **not** use BatchNorm. Batch Normalization was introduced later. AlexNet uses **max-pooling** for downsampling.

Which of the following statements about Residual Networks, ResNets, are correct?

a. Residual networks use skip-connections which add the representation h_l of a former layer l to a later layer l+1, that is: $h_{l+1}=h_l+F(h_l)$ where F can be any transformation in a neural network with appropriate dimensions, such as a fully-connected layer.

b. Because of skip-connections, very deep Residual Networks can be trained even without normalization techniques.

c. Leaky ReLUs cannot be used in Residual Networks.

d. Residual networks are always convolutional networks and cannot be used in networks that have only fully-connected layers.

e. Pre-trained Residual Networks cannot be used for transfer learning because the learned features are usually overly specified to the task on which they were trained.

a correct

Skip connections help a lot, but very deep ResNets usually still rely on normalization, especially BatchNorm.

Imagine you are working on a machine learning project related to self-driving cars. A camera inside a car provides images of road scenes. Your task is to train a neural network that supplies images in which cars, roads, pedestrians, etc. are marked in different colors.

a. Generative adversarial networks are the most appropriate type of networks to approach this task.

b. The intersection over union, IoU, would not be an appropriate metric to assess the performance of a model trained on this task.

c. This represents an object detection task in computer vision.

d. In order to train such a network, labels for each pixel of the training set images are required.

e. Deep neural networks cannot be used for this task because there is no appropriate loss function.

d correct

Assume you are given a convolutional neural network. You are only allowed to change the parameters of the last convolutional layer and you want to strongly increase the receptive field, RF, of each neuron in the last layer. For each change, decide whether they strongly increase, slightly increase, or do not increase the receptive field:

Switch to dilated convolutions Increase stride Increase the number of kernels Increase kernel size

- Switch to dilated convolutions: STRONG increase. Dilated convolutions spread the kernel over a wider spatial area.
- Increase stride of convolution: NO INCREASE. Increasing stride changes how far apart output positions are, but each individual output neuron still sees the same-sized patch of the previous layer.
- Increase number of kernels: NO INCREASE. More kernels mean only more output channels.
- Increase kernel size: SLIGHT increase. A larger kernel increases the receptive field, but only directly by the additional kernel coverage in the last layer. It usually increases RF less dramatically than dilation.

Which of the following models or architectures are one-stage and which are two-stage approaches?

Mask R-CNN, Faster R-CNN, R-CNN, Fast R-CNN, YOLO, SSD

1-Stage: YOLO, SSD. 2-Stage: All the R-CNNs

Which of the following statements about deep-learning-based object detection methods are correct?

a. Fast R-CNN has two output layers, also named heads: one for regression and one for classification.

b. The operations ROI-pooling and ROI align operate on feature maps and not on the input image.

c. Object detection can be formulated as regression task in a multi-task setting.

d. R-CNN uses AlexNet to extract features for Regions Of Interest.

e. Faster R-CNN uses a region proposal network rather than the selective search algorithm that was used in R-CNN.

a, b, c, d, e correct

Which of the following statements about autoencoders, not including variational autoencoders, are correct?

a. Autoencoders are always fully-connected networks.

b. Autoencoders do not map a single input data point to a single point in latent space, but they map a single input data point to a distribution in latent space.

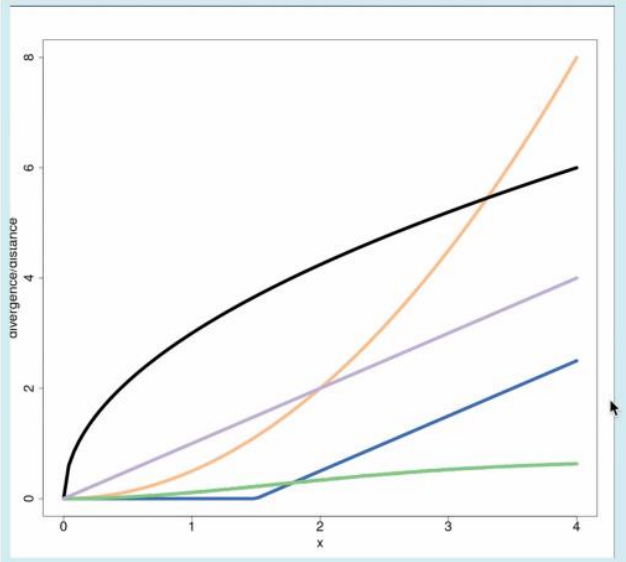
c. The Input-Output-Jacobian of an autoencoder is always a lower-triangular matrix.

d. Autoencoders consist of an encoder and a decoder, which are parametrized neural networks.

e. All autoencoders use fewer neurons in the hidden layers than in the input layer.

d correct

Assume you are given two univariate Gaussian distributions. The first one, p1 has mean zero and unit variance (N (0, 1)). The second Gaussian p2 has also unit variance, but a mean at x > 0, hence: N (x, 1). You are interested in the Kullback Leibler divergence KL (p1kp2), the Jensen-Shannon Divergence JSD (p1kp2), and the Wasserstein-2 distance WD (p1kp2) between these two distributions. You now increase x and plot this value against the divergence or distance measures, which gives three graphs displayed in the figure below. Which divergence belongs to which graph?



KL(p1‖p2) is quadratic, grows unbounded -> orange curve

W2(p1,p2) is linear -> purple straight line

JSD(p1‖p2) is bounded, approaches log 2 (saturates) -> green curve

Using the notation introduced in the lecture, which of the following loss functions are correct formulations of the standard Variational Autoencoder (VAE) loss function?

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x} \mid \mathbf{z}) - D_{KL}\left(q_\phi(\mathbf{z} \mid \mathbf{x}) || p_\theta(\mathbf{z})\right)$$
$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x} \mid \mathbf{z}) - D_{KL}\left(p_\phi(\mathbf{x}) || p_\theta(\mathbf{z})\right)$$
$$\mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[D(G(\mathbf{z})]$$
$$\mathcal{L}_{\boldsymbol{\theta}}(\mathbf{x}) = D_{KL}\left(p_{\mathbf{x}}(\mathbf{x};\boldsymbol{\theta}) \| p^*_{\mathbf{x}}(\mathbf{x})\right)$$
$$\mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))]$$

a correct

The standard VAE objective is the ELBO: reconstruction likelihood minus KL regularization between the approximate posterior qϕ(z|x) and the prior pθ(z). Options c and e are GAN losses, b compares unrelated distributions, and d is not the standard VAE loss.

Which of the following statements about variational autoencoders, VAEs, are true?

a. Variational autoencoders cannot be used to generate images.

b. The prior distribution of the variables z in the latent space must be a discrete distribution for VAEs.

c. Variational autoencoders have an encoder-decoder structure, where the encoder and decoder must be symmetric.

d. Variational autoencoders maximize a lower bound on the likelihood of the data.

e. Variational autoencoders automatically provide the marginal likelihood of a data point p(x) at low computational costs.

d correct

The latent prior $p(z)$ is usually a continuous distribution. VAEs do have an encoder-decoder structure, but they do not need to be symmetric. VAEs maximize the ELBO, a lower bound on the data likelihood. The exact marginal likelihood involves an integral, that is usually intractable, which is exactly why VAEs optimize a lower bound instead.

Using the notation introduced in the lecture, which of the following objective functions are correct formulations of the standard objective function of Generative Adversarial Networks (GANs)?

$$\min_G \max_D \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(D(G(\mathbf{z})))]$$
$$\min_G \max_D \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))]$$
$$\min_G \max_D \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log G(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - G(D(\mathbf{z})))]$$
$$\min_G \max_D \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{x}\sim p_r}[\log(1 - D(G(\mathbf{x})))]$$
$$\min_G \max_D \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[D(G(\mathbf{z}))]$$
$$\min_G \max_D \mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(\mathbf{z}))]$$

b correct

Here: x∼pr is a real data sample. z∼pz is latent noise. G(z) is a generated fake sample. D(x) is the discriminator's estimate that x is real. D(G(z)) is the discriminator's estimate that the generated sample is real. The discriminator wants: D(x)→1 for real samples, and D(G(z))→0 for generated samples. So it maximizes: logD(x)+log(1−D(G(z))). The generator wants to fool the discriminator, so it tries to minimize the same expression.

Which of the aspects listed below is most important to make GANs produce reasonable generated data?

a. Balancing learning speed of generator and discriminator

b. Regularising the input distribution of the discriminator

c. Balancing architecture of generator and discriminator

d. Regularising the weights of generator and discriminator

e. Regularising the input distribution of the generator

a correct

GAN training is a game between the generator and discriminator. Neither network should learn much faster than the other. If the discriminator becomes too strong too early, the generator gets weak or vanishing gradients. If the generator dominates, the discriminator gives poor feedback. So the learning dynamics must be balanced.

How can you use adversarial examples to generate new data?

a. it is impossible to generate data using adversarial examples, you need GANs

b. construct a static dataset with real and adversarial examples and train a generator with the data

c. apply adversarial attacks to a so-called generator model after every epoch

d. apply adversarial attacks to a so-called discriminator model after every epoch

e. construct a static dataset with real and adversarial examples and train a discriminator with the data

d correct

Adversarial generation works by attacking the discriminator: you modify inputs so that the discriminator is fooled into thinking they're real. You do not attack the generator; you use the discriminator's feedback to create better fake samples.

Consider a Generative Adversarial Network, GAN, which successfully produces images of apples. Which of the following statements are correct?

a. After training the GAN, the discriminator loss eventually reaches a constant value.

b. The discriminator can be used to classify images as apple vs. non-apple.

c. The generator can produce unseen images of apples.

d. The generator aims to learn the distribution of apple images.

c, d correct

In an ideal GAN equilibrium, the discriminator becomes unable to distinguish real from generated samples and its output approaches 0.5. Then the loss may stabilize. But in practice, GAN training is unstable and oscillatory. The discriminator is trained to classify **real apple image vs. generated fake apple image.** The generator aims to learn the underlying data distribution of apple images, so that generated samples resemble real apple samples.

Decide for each statement whether it is more likely for a GAN or a WGAN

The discriminator performs regression.

There are log-terms in the objective.

The objective is likely to suffer from the vanishing gradient problem.

Uses many more optimization steps for the discriminator than for the generator.

No weight-clipping is needed.

The discriminator performs regression.	WGAN
There are log-terms in the objective.	GAN
The objective is likely to suffer from the vanishing gradient problem.	GAN
Uses many more optimization steps for the discriminator than for the generator.	WGAN
No weight-clipping is needed.	GAN

Tick the correct statements related to the Inception Score, IS, and the Fréchet Inception Distance, FID.

a. IS and FID are metrics that assess the quality of GAN-generated images.

b. The FID computation is based on an explicit formula for the Wasserstein-2 distance between 2 Gaussian distributions.

c. Both IS and FID rely on features for input images that are produced by AlexNet.

d. The FID in general is better suited to detect mode collapse than the IS, since it takes into account the covariances.

e. IS and FID can be used as GAN objectives that lead to a faster training procedure.

a, b, d correct

FID compares real and generated image distributions in feature space by approximating them as Gaussian distributions. It then uses the closed-form Wasserstein-2 distance between Gaussians. IS and FID use an **Inception network**. FID compares both mean and covariance of real/generated feature distributions. This makes it better at detecting mode collapse. IS and FID are evaluation metrics, not training objectives.

Suppose that you have a fixed, multi-dimensional, non-Gaussian distribution p(x) that you want to approximate with a Gaussian distribution. You decide that you want to approximate this distribution with a Gaussian q(x) = N(μ, Σ) with some mean vector μ and some covariance matrix Σ. You now decide to minimize the Kullback-Leibler divergence between the two distributions KL(p || q) with respect to μ and Σ. Which expressions for μ and Σ minimize the KL-divergence between the two distributions?

a. $\mu = E(x^T x)$ and $\Sigma = I$ b. $\mu = E(x)$ and $\Sigma = Cov(x)$ c. $\mu = E(x^T x)$ and $\Sigma = E(x^T)$ d. $\mu = E(x)$ and $\Sigma = I$

e. $\mu = E(x)$ and $\Sigma = E(xx^T)$

b correct

Minimizing KL(p || q) makes q match the moments of p. So the best Gaussian approximation has the same mean and covariance as p: $\mu = E(x)$, $\Sigma = Cov(x)$. Option e is close, but $E(xx^T)$ is the second moment, covariance is $E(xx^T) - E(x)E(x)^T$.

What assumptions are implicitly made when using the BCE loss function for image reconstructions with a convolutional auto-encoder?

a. input pixels have the same distribution as the target pixels

b. input pixels are distributed according to a Bernoulli distribution

c. input pixels can have any value on the real line (centered around 0)

d. input pixels must have values between 0 and 1

e. input pixels are distributed according to a Gaussian distribution

b, d correct

Binary cross-entropy treats each pixel like a Bernoulli variable, so pixel values are interpreted as probabilities or binary values. Therefore the reconstruction targets should be in the range [0, 1].

What assumptions are implicitly made when using the MSE loss function for image reconstructions with a convolutional auto-encoder?

a. input pixels are distributed according to a Bernoulli distribution

b. input pixels are distributed according to a Gaussian distribution

c. input pixels must have values between 0 and 1

d. input pixels have the same distribution as the target pixels

e. input pixels can have any value on the real line (centered around 0)

b, e correct

MSE corresponds to assuming Gaussian reconstruction noise. Unlike BCE, it does not require pixel values to be binary or between 0 and 1; it naturally works with continuous real-valued outputs, often centered around 0.

The main difference between the frequentist and Bayesian approach is marginalization over parameters when we define the probability for a class label given the input x and dataset D. To give a more formal definition:

$p(y^{test} \mid x^{test}, D) = E_{w \sim p(w|D)} [p(y^{test} \mid w, x^{test})]$

Deep Ensembles try to approximate the full expectation with (weighted) averages. Which options are valid?

$E_{w \sim p(w|D)} [p(y^{test} \mid w, x^{test})] \approx$

a. $\approx 1/M \sum^{mM}_{=1} p(y^{test} \mid w_m, x^{test})$, where w_m denotes the weights of the m-th network of the ensemble.

b. $\approx p(y^{test} \mid w, x^{test})$

c. $\approx 1/M \sum^{mM}_{=1} \alpha_m \, p(y^{test} \mid w_m, x^{test})$ with $\alpha_m = \text{softmax}(z)_m$ for some random normal distributed vector z

d. $\approx 1/M \sum^{mM}_{=1} p(y_m{}^{test} \mid w, x_m{}^{test})$ where the subscript m of the test data denotes the m-th equally sized fraction of the full data.

e. $\approx \sum^{mM}_{=1} p(y_m{}^{test} \mid w, x_m{}^{test})$ where the subscript m of the test data denotes the m-th equally sized fraction of the full data.

f. $\approx \sum^{mM}_{=1} p(y^{test} \mid w_m, x^{test})$

a correct

A deep ensemble approximates the Bayesian expectation by averaging predictions from several independently trained models with different weights w_m. Option b uses only one model, d/e average over data splits instead of model weights, and f misses the normalization by M. Option c uses random weights and also has an extra 1/M.

You design a standard 2-layer fully connected neural network. All activations are sigmoids and your optimizer is stochastic gradient descent. You initialize all the weights and biases to zero and forward propagate an input x through the network. What is the output ŷ of the network?

a. 0 b. e^x c. 0.5 d. -1 e. 1

c correct

With all weights and biases initialized to zero, every pre-activation is 0. The sigmoid of 0 is 0.5. Since the output activation is also sigmoid, the network output is 0.5.

Consider the model defined in the question before with parameters initialized with zeros. W[2] denotes the weight matrix of the second layer. You forward propagate a batch of examples, and then backpropagate the gradients and update the parameters. Which of the following statements is true?

a. The entries of the weight matrix W[2] are all positive.

b. The entries of the weight matrix W[2] may be positive or negative.

c. The entries of the weight matrix W[2] are all negative.

d. The entries of the weight matrix W[2] are all zeros.

b correct

After the first forward pass, the hidden activations are all 0.5. The update of W[2] depends on the prediction error, labels, and learning rate direction. Depending on the batch labels, the gradient can lead to positive or negative updated weights.

Assume that you want to implement a new layer for a deep neural network that employs the following transformation $h^{l+1} = M\ \mathrm{softmax}(\beta M^T h^l)$,

where $h^l \in R^d$ is the input to this layer l, β is a positive hyperparameter to be set by the user, $M \in R^{d \times J}$ is a matrix of J patterns with d features provided by the domain expert, and p is defined as $p := \mathrm{softmax}(\beta M^T h^l)$. Which of the following statements about this layer are true?

a. h^(l+1) is equivariant to permutations of the columns of M.

b. p is equivariant to permutations of the columns of M.

c. This layer cannot be built in deep learning architectures because gradients, i.e. $\partial h^{l+1}/\partial h^l$, cannot be backpropagated.

d. h^(l+1) is invariant to permutations of the columns of M.

e. p is equivariant to permutations of the rows of M.

f. p is invariant to permutations of the columns of M.

g. p is invariant to permutations of the columns of M.

b, d correct

If the columns of M are permuted, the entries of p are permuted in the same way, so p is equivariant. But $h^{l+1} = M\ p$ stays the same because the same columns and weights are just reordered together, so h^{l+1} is invariant. The layer is differentiable, so backpropagation is possible.

You're given input x = (1, -2, -1, 2) and a kernel weight matrix w = (0, 1, 2, 3)

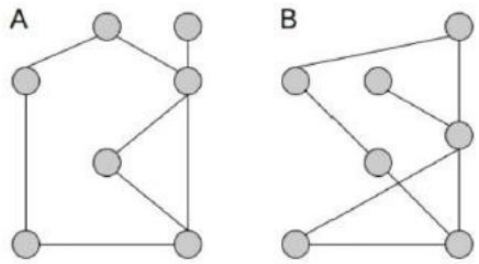
What is the result of the transpose convolution operation $*^T$, with stride 2, of w $*^T$ x = ?

a. [[0, 1, -1], [2, -1, -1], [-4, -2, 6]]	b. [[3, 2], [1, 0]]	c. [[0, 1, 0, -2], [2, 3, -4, -6], [0, -1, 0, 2], [-2, -3, 4, 6]]	d. [[0, 2, 1], [6, -14, 6], [2, 3, 0]]	e. [[1, 1, 2, 2], [1, 1, -2, -2], [2, 2, 4, 4], [2, 2, -4, -4]]	f. [[42]]	g. [[8, -5, 2], [-11, 5, 13], [19, 21, -29]]

c correct

With stride 2, each input value places a scaled copy of the kernel into the output, shifted by 2 positions.

Do the following two graphs fulfill the necessary condition for graph isomorphism, the Weisfeiler-Lehman test?



True

The graphs pass the WL necessary condition: their node-structure patterns can be matched. Both graphs have the same degree pattern, including one degree-4 node, one degree-3 node, one degree-1 node, and four degree-2 nodes. The neighborhoods around these nodes also match under relabeling, so the WL test does not distinguish them. The WL test does not find a difference, so the necessary condition for isomorphism is fulfilled.

Assume you are to design a CNN layer.

Input: [128, 128, 64], now adding a 1×1 Conv layer with 16 kernels, padding 0, stride 1.

a) Not possible to apply this conv layer

b) Resulting dim: [128, 128, 16]

c) 1×1 conv does not exist

d) Better: replace 1×1 layer with 2×2 max pooling → equivalent operation, but more efficient

e) Better: 1×1 with stride 2 → equivalent but more efficient

b correct

A 1×1 convolution is valid and keeps the spatial size when stride = 1 and padding = 0. It only changes the channel dimension: 16 kernels produce 16 output channels, so the result is [128, 128, 16]. Max pooling or stride 2 would reduce spatial resolution, so they are not equivalent. Using stride 2 would reduce spatial resolution (from 128×128 to 64×64). 2x2 max pooling also reduces dimensions to 64x64.

VGG Net:

a) Predominantly 3×3 conv

b) Single stage approach

c) Symmetric encoder-decoder structure

d) Majority of network weights located in fully connected layers

e) Cannot be used for image classification

a, d correct

VGG is known for stacking many small 3×3 convolutional layers. It was designed for image classification, not as an encoder-decoder segmentation architecture. A large share of its parameters comes from the fully connected layers at the end. “Single stage approach” is not the key VGG idea; that wording is more typical for object detection methods like YOLO/SSD.

Normalization for image classification:

a) BatchNorm used since AlexNet

b) Since Xception, most image classifiers started to use LayerNorm

c) BatchNorm used in InceptionNet to speed up training

d) BatchNorm part of most deep networks for image classification (e.g., ResNet)

c, d correct

AlexNet did not use BatchNorm; BatchNorm became important in later CNNs, including batch-normalized InceptionNet, because it stabilizes and speeds up training. BatchNorm normalizes each feature/channel using statistics from the mini-batch and is common in CNNs. LayerNorm is more typical for transformers and RNNs. LayerNorm normalizes across features within one sample. Common in transformers/RNNs.

Sparse Autoencoders:

a) Sparse AEs enforce sparsity on the representation via a regularization term.

b) Sparse AEs enforce hidden units to be inactive (i.e., activation = 0).

c) Sparse AE can be trained on a copy task, where input = target of network is identical.

d) Sparse AE force input units to be inactive (i.e., activation = 0).

e) Sparse AE enforce sparsity via early stopping.

f) Sparse AE cannot be trained on a denoising task, where input = noisy & target not.

a, b, c correct

Sparse autoencoders encourage most hidden/latent units to be inactive, usually via a sparsity penalty. They can still be trained as normal autoencoders on a copy/reconstruction task. Sparsity is applied to hidden representations, not input units, and it can also be combined with denoising.

Which of the following statements related to flow-based models are correct?

a. For Autoregressive flows the involved transformation g_ϕ is constructed in a way such that its Jacobi determinant always has value 1.
b. The computation of the Jacobi determinant of a neural network with D inputs and D outputs is in general of $O(D^3)$.
c. Fitting flow-based models is often done with the KL-divergence.
d. Flow-based models can represent any distribution $p_x(x) > 0$ if one starts from a sufficiently complex distribution. A simple base distribution $p_u(u)$, like the uniform distribution on the cube $(0,1)^D$, would not be sufficient.

b, c correct

Autoregressive flows are constructed so that the Jacobian is triangular, not so that its determinant is always 1. For a general dense $D \times D$ Jacobian matrix, computing the determinant has cubic complexity $O(D^3)$. Flow-based models avoid this cost by designing transformations with easy-to-compute Jacobian determinants, for example triangular, diagonal, coupling-layer, or autoregressive structures. Flow-based models are often trained by minimizing the forward KL divergence between the data distribution and the model distribution. A simple base distribution is usually sufficient in principle, as long as the flow transformation is expressive enough and invertible.

Which of the following examples represents a common group action used in graph NNs?

a) Rotation b) Scaling c) Translation d) Cropping e) Clustering

c correct

In graph neural networks, the key symmetry is usually permutation of node order. Reordering node indices should not change graph-level outputs and should only reorder node-level outputs correspondingly. Here “translation” is meant as this permutation/reindexing action, not spatial translation.

Translation (permutation) is most common group action in GNNs. GNNs are typically permutation invariant/equivariant, meaning that reordering nodes (translating indices) doesn't change the output. no inherent ordering of nodes.

Wasserstein distance / divergences:

a) JS & KL divergences show problems for distributions with disjoint supports — Wasserstein overcomes this.

b) A map D: S × S → R is a divergence if it is non-negative and symmetric.

c) Wasserstein-1 distance with Euclidean cost is not a divergence.

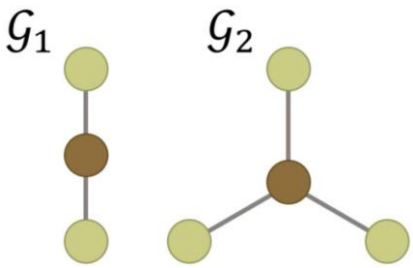
d) Wasserstein is defined as a minimization over joint distributions, but this is infeasible in practice.

e) Kantorovich-Rubinstein theorem allows formulating Wasserstein as a maximization over Lipschitz functions, making it tractable.

a, c, d, e correct

KL and JS can give bad or unusable gradients when distributions barely overlap, while Wasserstein still measures how much “mass” must be moved. A divergence does not need to be symmetric, so b is wrong (b describes a distance). Wasserstein is a distance, defined via an expensive optimization over couplings; the dual Lipschitz formulation makes it usable in WGANs.

Mark the correct statements concerning the two graphs below.

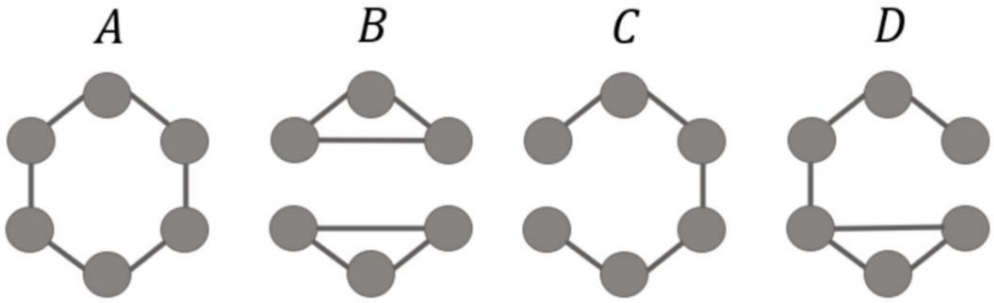


a. G_1 and G_2 can be distinguished by the Weisfeiler-Lehman color refinement test.
b. All message passing neural networks, MPNNs, can distinguish G_1 and G_2.
c. G_1 and G_2 are isomorphic graphs.

a correct

The graphs have different numbers of nodes. Also, their degree patterns differ. So WL can distinguish them. In principle, sufficiently expressive MPNNs can distinguish many graphs that 1-WL distinguishes, but not every possible MPNN architecture necessarily does so. A badly chosen or degenerate MPNN could map both graphs to the same representation. The graphs are not isomorphic.

Match the graphs with their corresponding adjacency matrix

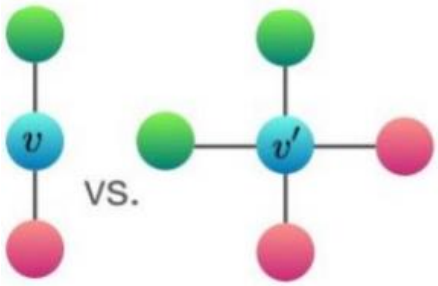


000011 000101 000110 011000 101000 110000	010000 101000 010100 001010 000101 000010	010001 101000 010100 001010 000101 100010	011000 101000 111000 000111 000101 000110	011000 101000 110100 001010 000101 000010

B C A None D

- Count the row sums of the adjacency matrix. Row sum = degree of that vertex.
- Check connected components. If the matrix is block-diagonal, the graph is disconnected.
- Check the diagonal. For ordinary simple graphs, the diagonal should be all zeros. A 1 on the diagonal means a self-loop.

In message passing neural networks, node v aggregates messages from its neighbors using a permutation-invariant aggregation function. Which aggregation functions can distinguish the two neighborhood multisets shown below?



a) Mean aggregation distinguishes them.

b) Max aggregation distinguishes them.

c) Sum aggregation distinguishes them.

d) Mean and Max both fail.

e) All common aggregation functions distinguish them.

c, d correct

Mean aggregation fails because both neighborhoods have the same average feature/color distribution. Max aggregation also fails because both neighborhoods contain the same set of feature types. But **sum aggregation can distinguish them**, because it counts multiplicities.

Let u be a random variable with density $p_u(u)$, and let x = T(u), where

T = [[2, -2],

[0, 1]]

What is the transformed density $p_x(x)$?

a) $2 \cdot p_u(Tx)$

b) $1/2 \cdot p_u(T^{-1}x)$

c) $p_u(Tx)$

d) $1/2 \cdot p_u(Tx)$

e) $2 \cdot p_u(T^{-1}x)$

b correct

For x = T(u), we need $u = T^{-1}x$. The change-of-variables formula is $p_x(x) = p_u(T^{-1}x) \cdot |\det(T^{-1})|$. Since det(T) = 2, we have $|\det(T^{-1})| = 1/2$. Therefore $p_x(x) = 1/2 \cdot p_u(T^{-1}x)$.

Autoencoder:

Encoder: $u(x) = \text{ReLU}(W_{enc}\, x)$

$W_{enc} =$ $\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$

Decoder: $\hat{x} = v(u) = \text{ReLU}(W_{dec}\, u)$

$W_{dec} =$ $\begin{bmatrix} 1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}$

Reconstruction loss: $L(x, \hat{x}) = \Sigma_i (x_i - \hat{x}_i)^2$

For input: $x = [1, 1, 2]^T$. Calculate the reconstruction loss.

Encoder:

$W_{enc}\, x = [-1, 1]^T$

$u = \text{ReLU}([-1, 1]^T) = [0, 1]^T$

Decoder:

$W_{dec}\, u = [0, -1, 0]^T$

$\hat{x} = \text{ReLU}([0, -1, 0]^T) = [0, 0, 0]^T$

Loss:

$L(x, \hat{x}) = (1 - 0)^2 + (1 - 0)^2 + (2 - 0)^2 = 6$

Mark the correct statments related to the Universal Approximator Theorem (UAT) for Neural Networks (NNs).

a. The UAT gives a useful algorithm to compute the optimal weights of a NN that should approximate a given continuous function.

b. In the case of a real-valued function h defined on $[-\pi, \pi]^2$, the existence of an approximating NN can be derived with the help of Fourier series.

c. The algorithm related to the UAT is easy to understand and implement, it is however very costly and therefore it is hardly used in practise.

d. The UAT shows that any function can be approximated by any given NN with arbitrary precision, if the weights are chosen appropriately. This means that there are no restriction on the function to be approximated and on the structure of the network that should do the job.

e. The UAT is an abstract statement about the expressive power of NNs.

b, e

The UAT is an existence theorem: suitable neural networks can approximate broad classes of functions arbitrarily well, but it does not give an algorithm for finding the optimal weights. Restrictions for the network: usually the function must be continuous on a compact domain, and the network class must have enough width/depth and a suitable nonlinearity.

Mark the correct statements related to Random Matrix Theory, RMT.

a. The quarter circular law is a statement about the behaviour of singular values for any given random matrix, no matter how the entries are distributed, if they are statistically independent or not, etc.

b. RMT provides information about the eigenvalues or singular values of large matrices with random variables as entries.

c. The empirical distribution of the eigenvalues of a symmetric matrix yields the average number of eigenvalues that lie in a certain region of the real line.

d. RMT only yields useful results when one works with square matrices.

b, c correct

Assume you are given an autoregressive model $p_w(x)$ for sequences (x_1, x_2, x_3). Each element of the sequence can be either A, B, or C.

The model $p_w(x)$ is given as follows:

$p_w(x_1) = \frac{1}{3}$ **for all** $x_1 \in \{A, B, C\}$ **and**

$$p_w(x_2 \mid x_1) = \begin{cases} \frac{1}{2} & \text{if } x_2 = x_1 \\ \frac{1}{4} & \text{if } x_2 \neq x_1 \end{cases} \qquad p_w(x_3 \mid x_2, x_1) = \begin{cases} \frac{1}{2} & \text{if } x_3 = x_2 \\ \frac{1}{4} & \text{if } x_3 \neq x_2 \end{cases}$$

You are given the sequence $x = (A, A, C)$. Calculate $p_w(x)$ and round the result to three digits after the comma.

$p_w(A,A,C) = 1/3 * ½ * ¼ = 0{,}333 * 0{,}5 * 0{,}25 = 0{,}042$

You design a standard 2-layer fully connected neural network with one hidden layer. All activations are sigmoids, and the optimizer is SGD. You initialize all weights and biases to zero and forward-propagate an input x. You train with MSE loss for a target y that is different from the model output at initialization.

Question: What happens when you perform backprop once?

a) The gradients with respect to all pre-activations are zero.
b) Neither weights nor biases are changed.
c) Only the output layer's weights and biases are updated; the first layer remains unchanged.
d) Only the biases in the network are updated.

c correct

The output pre-activation gradient is nonzero. Only the hidden-layer pre-activation gradient is zero. Only the output layer's weights and biases are updated after the first backprop step.

You are given a two-dimensional vector $u \sim p_u(u)$ and $x = T \cdot u$ with $T = \begin{pmatrix} 2 & 0 \\ 0 & \frac{1}{2} \end{pmatrix}$

Mark the correct formula for the distribution $p_x(x)$.

a) $p_x(x) = p_u\left(\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix} x\right)\right) \frac{1}{2}$

b) $p_x(x) = p_u\left(\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix} x\right)\right)$

c) $p_x(x) = p_u\left(\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{pmatrix} x\right)\right) \frac{1}{2}$

d) $p_x(x) = p_u\left(\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{pmatrix} x\right)\right)$

e) $p_x(x) = p_u\left(\left(\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} x\right)\right) 2$

f) $p_x(x) = p_u\left(\left(\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} x\right)\right) \frac{1}{2}$

Use the **change-of-variables** formula: $p_x(x) = p_u(T^{-1}x) \cdot |\det(T^{-1})|$

Equivalently: $p_x(x) = p_u(T^{-1}x) \cdot \frac{1}{|\det(T)|}$

Step 1 — Compute determinant

$$\det(T) = ad - bc = 2 \cdot \frac{1}{2} - 0 \cdot 0 = 1$$

Therefore: $\frac{1}{|\det(T)|} = 1.$ => There is no extra scaling factor.

Step 2 — Invert T

The inverse is: $T^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix}$

Therefore: $p_x(x) = p_u\left(\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix} x\right)\right)$